Mid-term sample paper — exam-style answers

These are **full answers**, written the same way as THEORY-NOTES.md.
When the notes already have a "Write this in the paper" line, that wording is used here so revision matches.

Questions: CV Mid Term Sample Questions.pdf.

Week practice in the same style:

- ANSWERS-Week1-Questions.md
- ANSWERS-Week2-and-Week3-Questions.md

Short answers (1–19)

1. Define a pixel in the context of digital images.

Imagine a chessboard. Each square is one **pixel** (picture element). Each pixel holds a number for brightness or color. The whole digital image is a **grid of these pixels**. A computer does not see a face or a tree. It only sees those numbers.

---

2. What is the difference grayscale and color image representation?

A **grayscale** image stores **one number per pixel**, usually 0 to 255. So it is a **2D matrix**: height × width. 0 is black, 255 is white, 128 is gray.

A **color** pixel needs **three** numbers: Red, Green, Blue. So a color image is a **3D matrix**: height × width × 3 channels.

Paper line: *A grayscale image is a 2D matrix of intensities 0–255. A color image is a 3D matrix with 3 channels. Origin is at the top-left. OpenCV uses BGR; Matplotlib uses RGB.*

---

3. Why is OpenCV's color order BGR instead of RGB?

OpenCV stores color as **BGR** (Blue, Green, Red). This came from older camera / Windows bitmap order, and OpenCV kept it.

Matplotlib and most websites use **RGB**. Same three numbers, different order. If you display an OpenCV image in Matplotlib without converting, **red and blue look swapped**.

---

4. What does the img.shape attribute return in NumPy?

It returns a tuple of sizes.

- Grayscale: `**(height, width)**`
- Color: `**(height, width, channels)**`, for example `(480, 640, 3)`

Height is rows, width is columns. That matches `image[y, x]`.

---

5. Define a feature in computer vision.

You cannot compare two photos pixel by pixel. If the camera moved 2 cm, almost every pixel changed, even if it is the same room.

So we pick **special places** that are likely to be found again: corners, blobs, distinctive textures. Those patterns are **features**.

A good feature is **distinctive** (not confused with other places), **repeatable** (found again after rotate / scale / light change), and **compact** (a short vector, not the whole image).

Paper line: *A feature is a distinctive, repeatable pattern. A keypoint marks its location. A descriptor encodes the local neighborhood as a vector so two images can be matched without comparing every pixel.*

---

6. What is a keypoint?

A **keypoint** is **where** the feature is — the (x, y) location, sometimes also scale and orientation.

Around that point we write a list of numbers that describe the neighborhood. That list is the **descriptor** (*what it looks like nearby*).

Keypoint = look here. Descriptor = what it looks like.

---

7. State one advantage of ORB over SIFT.

ORB is a faster, free alternative. It uses FAST to find corners and BRIEF as a binary descriptor, plus orientation. It is good enough for panorama / AR on a phone in **real time**.

SIFT is stronger when scale change is large, but it is **slow / heavy**. (It was patented; now free in modern OpenCV.)

Paper line: *SIFT is computationally heavy. ORB is a faster free alternative.*

---

8. What does pooling achieve in CNNs?

Pooling shrinks a feature map by summarizing small windows.

Max pool keeps the largest value (strongest detection). **Average pool** keeps the mean.

Benefits: smaller size, less computation, the network **tolerates small shifts**. The exact "where" becomes a bit fuzzy. That is useful after convolution has found the pattern.

---

9. Define ReLU and explain its purpose.

ReLU (Rectified Linear Unit): negative values become **0**, positive values stay. `ReLU(x) = max(0, x)`.

Purpose: it adds **non-linearity**. Without it, many layers would act like just one layer. It is also **faster than sigmoid** and **helps with vanishing gradients**, which is why AlexNet replaced sigmoid/tanh with ReLU.

---

10. What is dropout and why is it used?

Dropout is regularisation. While training, randomly switch off some neurons (say 30%). They output 0 for that step. The network cannot depend on one "star" neuron. Everyone must learn. That stops **overfitting** (memorising).

When you **test**, dropout is **off** — all neurons work.

Think of a team: random players sit out in practice, so the whole team knows the game.

---

11. State one key contribution of LeNet.

LeNet (1998, Yann LeCun) was a small CNN: conv → pool → fully connected, trained on **MNIST**, used for **bank cheques**.

Contribution: it **proved CNNs work** on images. It was a proof of concept that learning filters from pixels beats older hand-made features on digits.

---

12. What does "compound scaling" mean in EfficientNet?

Earlier models improved by only going deeper, or only wider, or only using bigger images. That wastes computation.

Compound scaling increases **depth, width and image resolution together**, using one scaling factor.

Result: high accuracy with fewer parameters. Works well on phones.

---

13. Define viewpoint variation in computer vision.

Viewpoint variation means the **camera angle changes the silhouette**. The same object looks different even though it did not change.

Example from the notes: a cup from the top is a **circle**; from the side it is a **U-shape**. The pixels change. A system must still recognise the cup.

---

14. What is occlusion, and why is it challenging?

Occlusion means **part of the object is hidden**.

Example: a person behind a car. The computer only sees a piece. Matching, detection and tracking can fail because the full pattern is not there. Tracking is also hard: the ID can be lost for a few frames and then swap with another person.

---

15. Define a GAN in simple terms.

A **classifier** says "this is a 7". A **GAN draws a new 7** that was never in the dataset.

GAN = Generative Adversarial Network (Goodfellow, 2014). **Two** networks with opposite jobs:

- **Generator G** (forger): random numbers `z` (noise) → a fake image
- **Discriminator D** (police): a real or fake image → a score, how real? (0 to 1)

They train as opponents with a **minimax** objective. At success, generated images match real data and D outputs about **0.5**. After training we keep the generator.

---

16. What dataset is used in the notebook?

The class GAN notebook uses **MNIST** — 28×28 handwritten digits 0–9. The same dataset LeNet used.

If your notebook file uses another set, write that name. The idea does not change.

---

17. What is the input to the Generator?

The Generator does **not** copy a real photo. Its input is a **random noise vector** `z` — a list of random numbers.

G turns that noise into an image. Different `z` should become different pictures. That is the source of variety.

---

18. What does the Discriminator output?

D outputs a **score from 0 to 1**: how real the image looks.

- **1.0** = "I'm sure it's real"
- **0.0** = "I'm sure it's fake"
- **0.5** = "I have no idea, it's 50-50"

---

19. Why do we usually keep only the Generator after training?

G is the **artist**. We want **new images**. D was only the teacher / police. Once G's pictures look real, D is not needed for generation. We throw D away and keep G.

Medium (20–32)

20. Explain the role of Hue, Saturation, and Value in color representation.

HSV is a color model that separates color from brightness.

- **Hue** — which color family (the angle on a color wheel: 0° red, 120° green, 240° blue)
- **Saturation** — how pure / washed (0% = gray, 100% = vivid)
- **Value** — how bright (0% = black, 100% = bright)

Why this split matters: a yellow banana in sun and in shade has very different RGB numbers, but **Hue stays yellow**. Only Value drops. So the computer can still detect yellow when lighting changes.

Exam definition: *Hue represents the color, Saturation represents the purity of the color, and Value represents the brightness. HSV is preferred for color detection than RGB because changes in lighting mainly affect the Value, while the Hue remains relatively stable.*

---

21. What is the purpose of a kernel in image processing?

A **kernel** is a small matrix (usually 3×3 or 5×5) that slides over an image. At each position we multiply overlapping values and sum them to get a **new pixel**.

Purpose: different kernels produce **blur, sharpen or edges**. The output is a feature map that highlights what the kernel cares about.

In CNNs, kernels are **learned automatically** to detect patterns.

---

22. How does histogram equalization enhance image contrast?

Suppose most pixels sit between 40 and 90. The photo looks dull because it is not using the full 0–255 range.

Histogram equalization remaps intensities so they use the full 0–255 range, which improves contrast. Darks get darker, brights get brighter. Details become easier to see. It is useful as a **preprocessing** step before thresholding or feature detection.

Limit: it can also **amplify noise**.

---

23. Explain the concept of descriptors with examples.

A **keypoint** marks a location. A **descriptor encodes the local neighborhood as a vector** so two images can be matched without comparing every pixel.

Examples:

- **SIFT:** a **128-dimensional** descriptor from local gradients. Robust to scale and rotation. Heavy.
- **ORB:** a **binary** string (BRIEF) with orientation. Fast and free. Weaker on large scale change.
- **HOG:** not a sparse keypoint method. It builds **histograms of edge directions in cells** for a whole window (pedestrian / face).

Same place in two photos → similar descriptor numbers. Different place → different numbers.

---

24. How does FLANN improve feature matching efficiency?

Matching compares descriptors of Image A and Image B to find the same feature points.

Brute force compares every descriptor to every other. Correct but **slow** when there are thousands of points.

FLANN (Fast Library for Approximate Nearest Neighbors) builds an **index** (like a smart search tree) and finds a **near** neighbor quickly. Much faster on large sets. There is a tiny chance the match is not the absolute best. For panoramas and real-time matching, the speed is worth it.

After matching, **Lowe's ratio test** still drops confusing pairs, and **RANSAC** drops outliers.

---

25. What is the role of RANSAC in robust estimation?

Some matches are still **wrong**.

RANSAC repeatedly tests different groups of matches and **keeps the transformation that fits the most correct matches**, while ignoring wrong ones (**outliers**). The good points are **inliers**.

The transform is often a **homography** (a 3×3 matrix that aligns one image plane to another). That is also how a logo detector draws a box in a scene.

Paper line: *RANSAC estimates that transform robustly by ignoring outlier matches.*

---

26. Explain the concept of skip connections in ResNet.

If you keep adding layers, training breaks. Gradients vanish and accuracy can get worse.

Skip (residual) connections: the input is copied forward and added to the layer's output:

output = F(x) + x

F(x) is the "change" the layer learns. x is the original input. Gradients can flow easily through the skip path, so very deep networks (50, 101, 152 layers) can be trained. If extra layers are useless, F can be near zero and the skip just copies x.

Paper line: *In ResNet they help gradient flow so deep nets can train.*

(If they also mention U-Net: same word, different job — U-Net copies high-resolution features to the decoder for sharp masks.)

---

27. How does YOLO differ from R-CNN in object detection?

R-CNN classifies **region proposals** with a CNN: guess many boxes, run a CNN on each. Accurate but **slow**. Faster R-CNN adds an **RPN** to propose regions, still two stages.

YOLO (You Only Look Once) looks at the **whole image once**. In that one pass it predicts all boxes and all classes together. **Real-time**. Weaker on **tiny** objects.

Choose: medical / tumour → R-CNN family (accuracy). Self-driving / live camera → YOLO (speed).

---

28. What is the role of the encoder and decoder in U-Net?

Draw this:

```
Encoder (down):  picture gets smaller → understands WHAT it is
Decoder (up):    picture grows back   → understands WHERE it is
Skip connections: copy encoder details into decoder → sharp edges
```

- **Encoder** = compress for **meaning / context** (this looks like tissue).
- **Decoder** = expand for **place** (these pixels).
- **Skips** = bring back the original edges that compression lost.

That is why U-Net is used in **medical** images: you need the exact boundary, not a box.

Paper line: *The encoder captures context, the decoder recovers location, skips bring fine detail.*

---

29. Explain how histogram equalization helps with illumination changes.

Illumination means light changes brightness and color. A face in sun vs in a room has different raw pixels, even though it is the same face.

Equalization **stretches** both photos toward the full 0–255 range. Contrast becomes more alike, so matching, thresholding or HOG later see a fairer image.

It does **not** fully fix a photo that is bright on one side and dark on the other. Then use **adaptive** methods. It can also boost noise.

This is one of the techniques used to survive the four core CV challenges.

---

30. Explain why normalization of MNIST images is important for GAN training.

MNIST images have pixel values from **0 to 255**. However, the **Generator (G)** usually produces images with pixel values in the range **–1 to 1** (using **tanh**) or **0 to 1** (using **sigmoid**).

If the **real images** an d **generated images** have **different value ranges**, the **Discriminator (D)** can easily tell them apart **just by checking the pixel values**, without looking at whether the digit actually looks real. This makes GAN training **unfair and unstable**.

Therefore, we **normalize** the real images so that they have the **same range** as the Generator's output. Now the Discriminator must compare the **actual image content**, not just the pixel scale.

---

31. Describe the two steps in each training round (update D, update G).

G does not copy a real photo. It only learns from D's "fake / real" feedback. Each round has **two opposite steps**.

Step 1 — update D. Show D real MNIST images (label real) and G's fakes (label fake). Train D to tell them apart. **G is frozen.** D is the inspector practising.

Step 2 — update G. Make new fakes from noise. Ask D to score them. Train G so D says "real". **D is not updated** in this step (or its weights are frozen). G is the forger practising.

Never train both as if they share one goal. D wants to be right. G wants D to be wrong. That fight is the **minimax** objective.

---

32. What is the role of random noise vectors in generating new images?

Each different noise vector `z` should become a **different** image. Noise is the **source of variety**.

The same trained Generator + a new random `z` = a new digit that was never in the dataset. If you always feed the same `z`, you always get the same picture. If G ignores `z` and repeats one digit, that is **mode collapse**.

Longer explain (33–48)

33. Describe the mathematical operation of convolution and its role in feature extraction.

Convolution is a sliding-window operation. A kernel moves across the image, computes a weighted sum of local pixels, and produces a **feature map**.

1. Put the kernel on the top-left of the image.
2. Multiply matching cells and add.
3. Store the result in the output.
4. Slide by a step (later called **stride**).
5. Repeat until the image is covered.

Formula:

[(I * K)(x, y) = \sum_m \sum_n I(x - m,\ y - n) \cdot K(m, n)]

`I` is the image, `K` is the kernel. It only says: look at neighbors, weight them with K, add.

Role in feature extraction: a blur kernel smooths, a sharpen kernel makes edges clearer, an edge kernel finds boundaries. In a **CNN** the same slide happens, but kernels are **learned**. Early layers find edges and blobs. Deeper layers find parts and objects. Convolution **is** how features are extracted. Fully connected layers later decide the class.

Paper line: *It is the foundation of both classical filtering and CNNs.*

---

34. Describe the difference between supervised and unsupervised learning with CV examples.

Supervised learning trains on examples that already have **answers (labels)**. Input + correct output. Later you predict the output for a new input.

CV examples: MNIST digits 0–9; photos marked cat / dog; boxes around cars (detection); pixel masks in U-Net. kNN and SVM are supervised.

Unsupervised learning has **only inputs, no labels**. The algorithm finds groups or structure by itself.

CV examples: **k-means** clustering of SIFT descriptors to build a **visual vocabulary** (the middle of Bag of Visual Words); a **GAN** learning the look of digits without class labels (unless it is a conditional GAN).

Fruit basket: supervised = you tell it "this is an apple". Unsupervised = it groups red-round things together without the word apple.

Paper line: *Supervised learning trains on labelled data to predict labels or numbers. Unsupervised learning finds hidden structure in unlabelled data, such as clusters.*

---

35. Describe how Vision Transformers use attention to capture global context.

A **CNN looks at a small window**. To connect the zebra's head to its tail you need many layers. **Attention** lets far parts **talk in one step**.

Attention uses three names:

- **Query (Q)** — what I am looking for
- **Key (K)** — what each piece is
- **Value (V)** — the information inside that piece

Compare Q with K → importance scores → mix the Values. Related pieces get high weight.

ViT steps:

1. Cut the image into **patches** (e.g. 16×16). Flatten each patch. Treat it like a **word**.
2. Send this list into a **Transformer** (attention).
3. A special **class token** summarises the whole image for the final label.

Patches keep local detail. Attention across patches gives a **global** view. ViT usually needs a lot of data.

Paper line: *This is how ViT sees global context that local CNN filters miss.*

---

36. Discuss how multi-scale feature extraction addresses scale variation in CNNs.

Scale variation: near looks big, far looks small. The same dog filling the frame vs in the distance. One window size fails.

Multi-scale means look at several sizes:

- an **image pyramid** (the same photo at many resolutions)
- **CNN feature maps of different depths** — SSD does this: shallow maps for small objects, deep maps for large ones
- several **box / anchor sizes**

A far pedestrian is a few pixels on a deep coarse map and is missed. A finer map still sees them. That is how multi-scale extraction addresses scale.

---

37. Explain the minimax objective function of GANs.

D wants to be **right**. G wants D to be **wrong**. One's win is the other's loss. That fight is the **minimax / adversarial / zero-sum** objective.

[\min_G \max_D \; \mathbb{E}*{x \|sim real}[\|log D(x)] + \|mathbb{E}*{z \|sim noise}[\log(1 - D(G(z)))]]

In words: D maximises the chance of scoring real images high and fakes low. G minimises that, which means G tries to make `D(G(z))` high (fool D).

How training goes in simple words: first G makes noise and D catches every fake. G gets a little better. D must look harder. If it works, G's pictures look real and D can only guess → score about **0.5**.

Paper line: *They train as opponents with a minimax objective. At success, generated images match real data and D outputs about 0.5.*

---

38. Discuss the three common problems in GAN training: mode collapse, instability, and D too strong.

This is not like a normal classifier where loss quietly goes down. **Both** nets are learning, so each one is chasing a **moving** target.

Mode collapse: G finds **one** picture (or one digit) that fools D and **repeats only that**. Each image looks fine, but **variety is gone**. D judges one image at a time, so the loss does not say "you only draw 1s".

Instability: loss jumps up and down. More training can make pictures **worse**. Small setting changes can break the run. **Do not trust the loss number.** Look at a grid of generated images.

D too strong: D becomes perfect early. G's gradient dies because `log(1 − D(G(z)))` is **flat** when D(G(z)) ≈ 0. G cannot learn. Fix idea: use the **non-saturating** generator loss `−log D(G(z))` so the slope stays useful; also slow D down (label smoothing, fewer D steps).

Paper line: *GAN training is a game, so the Nash equilibrium (G matches real data, D = 0.5) is hard to reach and training can oscillate. Mode collapse is when G produces only a few kinds of samples. Instability means losses and image quality swing. Loss value alone does not measure sample quality.*

---

39. Explain the process of thresholding and its types with examples.

Thresholding converts a grayscale image into a black-and-white (binary) image by choosing a cutoff T.

- If pixel > T → white (255)
- If pixel ≤ T → black (0)

Example: pixels 50, 120, 180, 240 and T = 127 become 0, 0, 255, 255. Gray shades are gone, so objects separate from the background.

Global: one T for the entire image. Works when lighting is uniform. Fails if one side is bright and one side is dark. Example: a well-lit scanned page.

Adaptive: a different T for each small region (bright area T = 150, dark area T = 80). Use when lighting is uneven. Example: outdoor fruit with shadow.

Otsu: you do not choose T. The computer automatically finds the best T that separates foreground and background. Best when the histogram has **two distinct peaks**, such as black text on a white page.

Used in document scanning, object detection, and medical image analysis.

Paper line: *Thresholding converts grayscale to binary. Global uses one cutoff and fails under uneven light. Adaptive uses local cutoffs. Otsu automatically chooses T by minimizing intra-class variance.*

---

40. Discuss morphological operations (dilation, erosion, opening, closing) with diagrams and applications.

Morphological operations modify the shape of objects, mainly on **binary** images. White = object, black = background.

They use a **structuring element** (a small 3×3 square, circle, or cross) that slides over the image and decides how each pixel should change based on its neighbors.

Exam definition: *Morphological operations are image processing techniques used to modify the shape of objects in an image. They use a small structuring element (kernel) that slides over the image to expand, shrink, or clean objects, and are mainly applied to binary images.*

Think of a marker and an eraser:

Dilation — grow the object. Makes white bigger. Closes small gaps and connects broken parts. Example: connect broken letters.

Erosion — shrink the object. Makes white smaller. Removes tiny white dots (noise). Example: thinning lines or stray dots.

Opening — clean the object. Erosion then dilation. First remove small noise, then restore the main object. Example: clean a noisy scan.

Closing — repair the object. Dilation then erosion. First fill small holes, then restore size. Example: fill holes and reconnect broken characters.

In the paper, sketch a small white blob and a 3×3 plus-sign: dilation adds a ring, erosion eats a ring. Opening vs closing is a favourite compare — **remember the order**.

Applications: document processing (noise, readable text); medical imaging (cells, tissues); industrial inspection (defects); object detection (clean shapes before contours).

Paper line: *Dilation expands foreground, erosion shrinks it. Opening = erosion then dilation (remove noise). Closing = dilation then erosion (fill holes).*

---

41. Explain Harris Corner Detection — method, pros, cons, and applications.

A **corner** is where intensity changes in **two** directions. An **edge** changes in only one. A **flat** wall changes in none.

Method: Harris looks at **gradients in a window** and scores "how corner-like". It builds a small matrix from those gradients and uses a corner score **R** (from the eigenvalues). High R → corner.

Pros: classic, finds corners well on buildings and chessboards. Fairly ok under rotation of a flat scene.

Cons: not invariant to scale or rotation in the strong SIFT sense. If you zoom, the window no longer matches. Sensitive to noise and to the threshold you pick.

Applications: starting points for **tracking** (KLT likes corners), **panorama** matching, older SLAM / camera pose.

Paper line: *Harris detects corners but is not scale/rotation invariant.*

---

42. Discuss the role of convolution, pooling, and activation functions in CNNs with examples.

A normal fully connected net has two problems with images: **too many numbers**, and it **forgets layout**. A CNN fixes this with a small filter that slides and **reuses the same weights** (weight sharing).

Convolution: the filter extracts one pattern. The output is a **feature map** — where that pattern appears. Example: first conv finds a vertical edge; later convs use that to find an eye.

Activation (usually ReLU): after conv, negatives become 0. Adds a **bend**. Without it, many layers act like one layer. Faster than sigmoid, helps vanishing gradient.

Pooling: shrinks the map (max pool keeps the strongest value). Less compute, tolerates small shifts.

Typical flow: Input → Conv → ReLU → Pool → … → Flatten → Fully Connected → class scores.

Early layers: edges, blobs. Deeper layers: parts, then objects. *Conv extracts, Dense decides.*

Paper line: *CNNs use convolution so filters detect local patterns with few parameters. Pooling downsamples feature maps. ReLU adds non-linearity.*

---

43. Explain the evolution of CNN architectures from LeNet to VGG, highlighting their key innovations.

Each architecture was built to **fix a problem** from the one before it. If you remember the problem, you remember the model.

LeNet (1998), Yann LeCun. A very small CNN: conv → pool → fully connected. Trained on **MNIST**. Used for bank cheques. **Innovation:** proved CNNs work better than older methods on images. Proof of concept.

AlexNet (2012). 5 conv + 3 fully connected, trained on **GPUs**, won **ImageNet** by a huge margin. **Innovations that stayed: ReLU** (faster, less vanishing gradient), **Dropout** (less memorising), **GPU**. Problem solved: deep nets at **large scale**. After this, everyone used deep CNNs.

VGG (2014). 16 or 19 layers, simple design: stack many **3×3** filters. Two 3×3 cover about the same area as one 5×5, with fewer parameters and more ReLUs. Problem solved: go deeper **without a messy design**. Limitation: still very heavy.

(If they also want the next names: ResNet = skip `F(x)+x` *so 100+ layers train. EfficientNet = compound scaling.)*

Paper line: *LeNet proved CNNs on digits. AlexNet introduced ReLU, dropout and GPU training and started the deep-learning boom. VGG showed depth with stacked 3×3 filters.*

---

44. Discuss the importance of ReLU and Dropout in training deep networks.

ReLU: negatives become 0, positives stay. Adds non-linearity. **Gradient is 1** for positive values, so the signal does not die like it does in sigmoid. Deep nets can train. Faster. This is why AlexNet dropped sigmoid/tanh.

Dropout: random neurons set to zero **during training** so the model does not overfit. The net cannot depend on one star neuron. At **test** time dropout is off.

Together: you can train a **deeper** network (ReLU keeps gradients alive) that still **generalises** to new photos (dropout). These two ideas from AlexNet stayed in almost every later net.

---

45. Describe the four core challenges in computer vision (viewpoint, illumination, occlusion, scale) with examples.

A computer must recognise the **same object** when the pixels change. Four reasons the pixels change:

Viewpoint. Camera angle changes the silhouette. A cup from the top is a circle; from the side it is a U-shape.

Illumination. Light changes brightness and color. A face in sun vs in a room.

Occlusion. Part of the object is hidden. A person behind a car.

Scale. Near looks big, far looks small. The same dog filling the frame vs in the distance.

Hand-crafted features (SIFT) and later CNNs are both attempts to survive these changes.

Paper line: *Core CV challenges are viewpoint variation, illumination changes, occlusion and scale variation. A robust system must recognise the same object under these changes.*

---

46. Describe the analogy of the Generator as a forger and the Discriminator as an inspector.

Generator G = forger. It starts from random numbers (noise), not from a copied note. It tries to draw a picture that looks like a real training image (a "real note").

Discriminator D = inspector / police. It sees real notes and fakes, and scores how real they look (0 to 1).

Each round the forger gets a little better, so the inspector must look harder. At the start D catches every fake. If training works, G's pictures look real and D can only **guess** (score about 0.5). G never traces a real photo. It only learns from D's "fake / real" feedback.

After training we keep the forger and throw the inspector away.

---

47. Explain the architecture of the Generator in the notebook (layers, activations, output).

In the usual **MNIST** class notebook, G is a **fully connected decoder** (not a photo copier).

- **Input:** noise vector `z` (about 100 random numbers).
- **Hidden layers:** Dense layers with **ReLU or LeakyReLU**. Sizes grow (for example 256 → 512 → 1024) so the net has room to build a digit.
- **Output: 784** numbers (28×28), with **tanh** or **sigmoid** so pixels sit in a known range.
- Those 784 values are **reshaped** into a digit image.

G does not take a real image as input. Only noise.

If the notebook is a **DCGAN** instead: G uses **transposed convolutions** to grow a small map into 28×28, often with **BatchNorm**. Same job, convolutional design. Sharper digits.

Write whatever layers **your** notebook shows. The exam point is: noise in, image out, last activation matches the normalised pixel range.

---

48. Explain the architecture of the Discriminator in the notebook (layers, activations, output).

D is a **binary classifier**: real vs fake.

- **Input:** a flattened 28×28 image (784 numbers), either a real MNIST digit or G's fake.
- **Hidden layers:** Dense + **LeakyReLU**, sizes shrink (for example 512 → 256).
- **Output: 1** number with **sigmoid** — probability that the image is real.

If the notebook is **DCGAN**: D uses **strided convolutions** to shrink the image to that score, like a small CNN classifier.

D sees **both** real images and G's fakes. After training we usually throw D away.

Compare / discuss (49–52)

49. Compare RGB and HSV color models — advantages, limitations, and use cases.

RGB creates colors by mixing Red, Green and Blue light. Every screen pixel uses these three. It is **excellent for displaying** images and for feeding CNNs (cameras and datasets are RGB/BGR).

Limit of RGB: the same object changes numbers when lighting changes. Yellow banana in sun `(255, 255, 50)` vs shade `(120, 120, 20)`. Color detection with a fixed RGB range **misses** objects.

HSV stores Hue (color family), Saturation (purity), Value (brightness). It **separates color from brightness**.

Advantage of HSV: when lighting changes, Value changes the most; **Hue stays relatively stable**. So HSV is preferred for **color detection** — ripe fruit, traffic lights.

Limit of HSV: Hue **wraps around** (red sits near 0° and 180° in OpenCV), so red may need two ranges. It is not how screens emit light, so you still **store and display** in RGB/BGR and **convert** when you search for a color.

Use RGB to store, display, and train CNNs. **Use HSV** to detect a color under changing light.

50. Compare Haar cascades and HOG for face detection — methodology, strengths, weaknesses, and applications.

Face detection = find *where* a face is (a box). It does **not** tell you whose face it is.

Haar cascade (Viola–Jones). It looks at simple **light/dark patterns** (eyes darker than cheeks, nose bridge brighter). The image is scanned in small windows. Each window goes through a **cascade** of quick checks. If it fails early, it is thrown out. **AdaBoost** picks which checks are useful. **Why fast:** most of the image is clearly not a face and dies in the first cheap steps. **Good at:** faces looking straight at the camera, normal lighting. **Bad at:** tilted or side faces, masks, strange lighting.

HOG. It looks at the **direction of edges** (gradients), so it captures **shape / outline**, like head and shoulders. It does not depend much on brightness, so it **handles lighting better than Haar**. **Downside:** slower, more compute. Also used for **pedestrians**. Sensitive to **scale** (the person must be roughly the expected size).

Paper line: *Haar-cascade detection uses rectangular Haar features selected by AdaBoost and a cascade classifier. It is fast for frontal faces but sensitive to pose and lighting. HOG captures gradient orientation histograms, so it is more robust to illumination but computationally heavier.*

Today many apps use CNNs, but the paper still wants this compare.

---

51. Compare R-CNN, YOLO, and SSD architectures — processes, strengths, weaknesses, and ideal applications.

R-CNN (2014). Guess many possible boxes, run a CNN **on each box**, say the class and fix the box. **Plus:** accurate. **Minus:** very slow (thousands of CNN runs). **Faster R-CNN:** a small **RPN** learns the boxes. Still two stages: first propose, then classify. **Ideal:** medical / tumour — accuracy matters more than speed.

YOLO (2016). Looks at the **whole image once**. Predicts all boxes and classes together. **Plus:** real-time. **Minus:** weak on **tiny** objects. **Ideal:** driving, sports, live CCTV.

SSD (2016). Also one shot. Looks at **several feature maps of different sizes**, so it catches **small and large** objects. Middle of speed and accuracy. **Ideal:** phones, mixed object sizes.

Paper line: *R-CNN classifies region proposals with a CNN: accurate but slow. Faster R-CNN adds an RPN to propose regions. YOLO predicts boxes and classes in one forward pass: real-time, weaker on small objects. SSD is also single-shot and uses multi-scale feature maps. Choose R-CNN when accuracy matters more than speed; YOLO/SSD when real-time is required.*

52. Explain techniques used to overcome these challenges — data augmentation, histogram equalization, robust feature detection, and multi-scale feature extraction.

The four challenges are viewpoint, illumination, occlusion, scale. The question names four techniques. Tie each technique to the challenge.

Data augmentation. You randomly rotate, flip, crop, scale the training images. The CNN sees many viewpoints and scales of the same object, so it does not memorise one pose. This mainly helps **viewpoint** and **scale**.

Histogram equalization. Remaps intensities to the full 0–255 range. A dark photo and a bright photo of the same room become more alike. This helps **illumination**. Limit: can boost noise; uneven light may still need adaptive methods.

Robust feature detection. SIFT / ORB keypoints + descriptors + **RANSAC**. You match distinctive points, not every pixel. Occluded or wrong matches become **outliers** and are ignored. SIFT also survives scale and rotation. This helps **occlusion**, **viewpoint**, and some **scale**.

Multi-scale feature extraction. Image pyramid, or several CNN maps (SSD), or SIFT's Gaussian scale space. The same object is looked at in more than one size. This is the direct answer to **scale variation**.

Paper line to close: *A robust system must recognise the same object under viewpoint, illumination, occlusion and scale changes.*

Elaborate (53–57)

53. Discuss the training loop in detail: how labels_real and labels_fake are used, and how gradients flow in each step.

Take a batch of real MNIST images `x` and a batch of noise `z`.

Update D (inspector practises).

`labels_real` is 1 (sometimes 0.9 — **label smoothing**, so D does not become over-confident). `labels_fake` is 0.

D sees real `x` and should match `labels_real`. D sees `G(z)` and should match `labels_fake`. The loss is "be right on both". **Gradient goes into D only.** G is frozen, so G is not punished for D getting better. D becomes a harder exam.

Update G (forger practises).

Make new `G(z)`. Pass those fakes through D. Now the labels on those fakes are the **real** labels (1). G's loss is "make D say real". **Gradient flows from D back into G.** D's weights are **not** updated in this step. If you also stepped D here, D would become worse on purpose and the game breaks.

These two steps are the practical form of the **minimax** fight. Repeat many times. Watch a **grid of images**, not only the loss number.

---

54. Elaborate on contour analysis and object tracking — algorithms, challenges (occlusion, scale, clutter), and real-world applications.

Contour analysis. After thresholding, a white blob has a boundary. That boundary curve is a **contour** — a curve joining continuous boundary points of the same intensity.

From a contour you can compute:

- **Bounding box** — smallest upright rectangle; localises "the object is here"
- **Convex hull** — rubber band; smallest convex polygon; no dents
- **Area, perimeter, centroid** — classify big vs small, round vs long; centroid for "where"
- **Douglas–Peucker** — simplify to a few corners; 3 = triangle, 4 = rectangle, many = circle

Object tracking. A photo is one moment. A video is many photos in a row. Tracking means keep finding the **same** object in every next frame.

Three approaches:

1. **Detection-based** — run YOLO / Faster R-CNN every frame, then **link** boxes (SORT / DeepSORT). Accurate, heavy.
2. **Tracking-by-motion** — follow pixels or a box with **optical flow** or KLT. Faster. Can lose the object if it hides or jumps.
3. **Hybrid** — detect sometimes, **predict** in between. Kalman filter: predict the next position, then correct with a measurement.

Challenges: occlusion (object hidden — ID lost), **scale** (near/far — box size changes), **clutter** (many similar objects swap IDs), messy background.

Applications: CCTV / surveillance, self-driving cars, sports replay, a robot following a person.

Paper lines: *A contour is a curve joining continuous boundary points of the same intensity. Bounding boxes localise objects. Object tracking means keep finding the same object in every next frame.*

---

55. Elaborate on the Bag of Visual Words pipeline — feature extraction, clustering, histogram representation, and classification.

Before CNNs, we extracted features by hand, then gave a vector to a classifier. **Bag of Visual Words** is like a word-count for images.

1. Feature extraction. From many training images, extract local descriptors (**SIFT** or **ORB**) at keypoints.

2. Clustering. Run **k-means** on all those descriptors. Each cluster center is a **visual word**. Together they are the **vocabulary**. This step is **unsupervised**.

3. Histogram representation. For a new image, the computer checks **which visual words are present** and **how many times each one appears**. The image becomes a histogram of visual words. Order is ignored — that is the "bag".

4. Classification. That histogram is given to **SVM** or **kNN**, which predicts the class (cat, dog, car). SVM and kNN are **supervised**.

Paper line: *Bag of Visual Words clusters local descriptors into a vocabulary and represents each image as a histogram of those visual words.*

---

56. Elaborate on how modern architectures (ResNet, EfficientNet, ViT) solved depth and efficiency challenges, and analyze their impact on real-world vision tasks.

The problem after VGG: stacking more 3×3 layers makes the net **too deep to train** (vanishing gradient, degradation) or **too heavy** to run on a phone.

ResNet (2015). Skip connections: **output = F(x) + x**. Gradients flow through the skip. 50 / 101 / 152 layers train. **Impact:** default **backbone** for classification, medical transfer learning, Faster R-CNN and Mask R-CNN. The skip idea also appears in U-Net (for detail, not for depth).

EfficientNet (2019). Compound scaling — grow depth, width and resolution **together**. High accuracy with fewer parameters. **Impact: mobile / on-device** vision. You do not need a huge VGG-style net on a phone.

ViT. A CNN is a magnifying glass. Attention lets far patches talk in **one step**. The image is cut into **patches**, treated as words, sent through a transformer. **Impact:** strong when you have **a lot of data** (or pretraining). Opened the door to vision-language models. Needs more data than a CNN of the same size.

Together they solved "we cannot go deeper" (ResNet), "deeper/wider/bigger alone wastes compute" (EfficientNet), and "local filters miss global context" (ViT). Real systems mix them: EfficientNet or ResNet on a phone or as a detector backbone; ViT when data and compute allow a global view.

---

57. Analyze the mathematical foundation of GANs: minimax game, optimal Discriminator, Nash equilibrium, and why the non-saturating generator loss is used.

Minimax game. D maximises being right. G minimises D being right:

[\min_G \max_D V(D,G) = \mathbb{E}*{x}[|log D(x)] + |mathbb{E}*[\log(1 - D(G(z)))]]

This is adversarial / zero-sum: one's win is the other's loss.

Optimal Discriminator. For a **fixed** G, the best D is

[D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}]

If G already matches real data (`p_G = p_data`), then *D = 1/2**. That matches the simple story: D can only guess, score about 0.5.

Nash equilibrium. Neither player can do better by changing **only themselves**. For a GAN that means: G's images match real data, and D answers **0.5** for everything. In real training they often **circle** this point instead of landing on it.

Why non-saturating generator loss. Early on, D is strong and `D(G(z)) ≈ 0`. Then `log(1 − D(G(z)))` is **flat**. G gets almost **no gradient** and cannot learn (D too strong).

So G instead **maximises** `log D(G(z))`, that is minimise `−log D(G(z))`. Same goal (fool D), **steeper slope** at the start. That is the loss used in most notebooks.

Paper line: *GAN training is a game, so the Nash equilibrium (G matches real data, D = 0.5) is hard to reach. Mode collapse and instability are why you look at samples, not only loss.*

Close this file. Write question 45 and question 51 on paper without looking. Those two recycle the most THEORY-NOTES wording.